HAZMAT SAFETY DATA SHEET

0x3212b29E33587A00FB1C83346f5dBFA69A458923 (verified as IMBTC)

Read 22,174 bytes of deployed bytecode and checked it against 16 registered hazards. 1 hazard is present, reproduced against this very contract.

What this is: It is a plain contract on mainnet, read from its bytecode.
What matched: 1 of the 16 registered hazards is present: HZ-016 (asset-handling).
How sure: HZ-016 was reproduced against this exact contract, not a lookalike.
The sharpest one: HZ-016: A token that calls the recipient back mid-transfer drains a counterparty that settles after paying
What is at stake: up to the credited balance a naive counterparty is holding; about 25M USD in one event against imBTC in 2020.
Precedent: Lendf.Me, 2020-04-19, roughly 25M USD, re-entrancy through imBTC's ERC777 callback.
What to do about it: Not a token bug: any contract that accepts this token must clear its credit before paying out, or re-enter-proof its payout path. Where the counterparty cannot be changed, deposit through a wrapper that uses transfer() and keeps the callback off the hot path.

ASSERTION LEVEL: PATTERN MATCH ONLY; CONFIRMATION RUNS ARE SEPARATE
A match is a candidate, not a clearance: this sheet fingerprints a contract against registered hazard shapes, it does not audit it. A clean sheet is not the same as safe.

HAZARDS FOUND

HZ-016 A token that calls the recipient back mid-transfer drains a counterparty that settles after paying
class 6 asset-handling CONFIRMED ON THIS TARGET
This token is an ERC777: transferring to a contract calls that contract back before the transfer returns, resolved through the ERC1820 registry, and the ERC777 send() additionally requires a contract recipient to have registered an implementer. A counterparty that credits on deposit and only clears the credit after paying out can therefore be re-entered by the recipient, paid twice out of one balance, and left unable to pay the next honest depositor. The token is not at fault and is behaving to spec. imBTC was the token in the Lendf.Me drain of 2020-04-19, roughly 25 million dollars in one re-entrancy against exactly this shape, which is why a token whose selector set says ERC777 is worth knowing about before it is accepted, not after.
What is at stake: up to the credited balance a naive counterparty is holding; about 25M USD in one event against imBTC in 2020.
What to do: Not a token bug: any contract that accepts this token must clear its credit before paying out, or re-enter-proof its payout path. Where the counterparty cannot be changed, deposit through a wrapper that uses transfer() and keeps the callback off the hot path.
Reproduced here: confirmations/HZ-016-3212b29e.json - a rebuild of the verified source and the probes in it passed. Confirming behaviour, not intent.
Lendf.Me, 2020-04-19, roughly 25M USD, re-entrancy through imBTC's ERC777 callback.
matched by selectors: 0x9bd9bbc6 0x62ad1b83 0x06e48538 0x556f0dc7 0x959b8c3f 0xd95b6371

WHAT WAS READ
a match means the selectors a placard asks about are present. It does not mean reachable, and a clean sheet is not a clearance.
placards checked 16 classes 7 selectors in code 42 the register asks about 12 otherwise unaccounted 30
0x06fdde03 name()
0x095ea7b3 approve(address,uint256)
0x23b872dd transferFrom(address,address,uint256)
0x2d07953a distributeRevenue()
0x3092afd5 removeMinter(address)
0x313ce567 decimals()
0x3ba0b9a9 exchangeRate()
0x3e9491a2 revenue()
0x45338d63 setRevenueAddress(address)
0x5c975abb paused()
0x75ab9782 tokensToSend(address,address,address,uint256,bytes,bytes)
0x8f32d59b isOwner()
0x92ff0d31 transferable()

0x95d89b41 symbol()
0x983b2d56 addMinter(address)
0xa9059cbb transfer(address,uint256)
0xaa271e1a isMinter(address)
0xaabbb8ca getInterfaceImplementer(address,bytes32)
0xb187984f disableTransfer()
0xb410908d revenueAddress()
... and 4 more

NOT CHECKED AT ALL
- resolution is best effort: unknown proxy patterns, diamond facets and beacons pointing at beacons
- are labelled not followed
- reachability
- economic preconditions
- state at any particular block
- source-level reasoning
- anything not covered by a placard

THE TECHNICAL PART
target 0x3212b29E33587A00FB1C83346f5dBFA69A458923
family evm
code sha256 0xddb50063c72473b19188a335c6c7a6072b2899657c980147d37045fccb9c0603
body bytes read 22174
metadata excluded 43 bytes (CBOR trailer, appended after compilation)
four-byte constants 42
register 16 placards v0xd7dea67cdf60b0bc
anchor epoch 3 root 0xed6a388e080b234215fcd1d0fc845207714daece4261c0ff9d7e39ce45c5a26c (epoch-3.json)
register digest 0xd7dea67cdf60b0bc
anchor root 0xed6a388e080b234215fcd1d0fc845207714daece4261c0ff9d7e39ce45c5a26c
selector index 783 names, a convenience reading and not verification

Cite the placard id, the code sha256 and the register digest. The HTML sheet is the artifact that carries its own digest; this PDF is the sam